

Analysis Question

1a. How many encryption and decryption operations occur before A receives the new key?
[2.5 pt]

Four encryptions and four decryptions occur before A receives the new key.

1b. Which entities perform each encryption and decryption operation in the process?
Describe the sequence of operations.

First, GSC encrypts the key using the shared G^{2A} key and then sends it to Group 2a. From here Group 2a decrypts the key using the shared G^{2A} key and then it encrypts it using shared G^{3A} key sending it to Group 3a. From here Group 3a, decrypts using the shared G^{3A} key and then sends it to Group 3B after encrypting it with the shared G^{3B} key. Lastly, Group 3b receives and decrypts the key using the shared G^{3B} key and then encrypts it again using the same key before sending to A. Since A is within the group it decrypts the key with the shared G^{3B} key as well.

2a. Identify all the keys that must be changed to maintain forward secrecy. Explain why each key needs to be updated

K7, K78, K58, and K18 all need to be changed to maintain forward secrecy. These keys need to be updated because R7 knows all of these keys. K7 is its personal key; K78 is the shared key between R7 and R8. Continuing up the tree, K58 is the shared key between R5 through R8. Lastly, the root key, K18, is the shared key between R1 through R8. So, all of these keys will need to be changed to maintain secrecy.

2b. Describe one efficient way to distribute the new keys to the remaining group members. Include the sequence of key encryptions and which members receive each encrypted key.

First, we should encrypt K78 using R8's personal key K8 this is because R7 does not have this key and will not be able to decrypt. Then, with the new K78 send it only to R8. For the new K58, send it through the old K56 to R5 and R6, but the new K78 where only R8 will receive the key. Lastly, encrypt a new K18 and send it out using the new K58 to R5, R6, R8 and old K14 to R1, R2, R3, R4. This will make a new root key that every member can access except R7.

On TGDH Security and Efficiency

1. What is the computational cost (in terms of modular exponentiations) for a single member to compute the group key after a Join event? Express your answer in terms of the tree height h .

The computational cost for a single member after a Join event is $O(h)$. This is because the new member must climb up the tree from its leaf to the root in order to compute the final group key. At each level, a new intermediate key is computed, and each of these computations requires one modular exponentiation. Since the number of levels that must be climbed is equal to the height of the tree, the total cost depends on h . For a balanced binary tree with n members, the height is $h = O(\log_2 n)$. Expressed in terms of n , the cost is $O(\log n)$.

2. In standard (two party) Diffie-Hellman, the Decisional Diffie-Hellman (DDH) assumption ensures that g^{ab} is computationally indistinguishable from a random group element given g^a and g^b . Discuss how the security of TGDH relies on the DDH assumption at each level of the tree. Is a single DDH assumption sufficient, or does TGDH require a stronger assumption?

A TGDH relies on the DDH to randomize one level. So, the DDH will take the public key (pk) and do exponentiation with the level's secret key (sk_n), $p^{(sk_1 \times sk_2)}$. This ensures randomization for this level alone. So, when we have two levels of DDH $p^{(sk_1 \times sk_2)}$, $p^{(sk_3 \times sk_4)}$ and we combine them without having another DDH, there is a security risk. If one level is discovered by the adversary, then half of the upper level's randomness is lost. Meaning that the security of the upper level is contingent on the lower-level keys staying random. In order to be sure the upper level must have a DDH assumption as well, $p^{(sk_1 \times sk_2 \times sk_3 \times sk_4)}$. Therefore, a single DDH assumption is not sufficient because the TGDH requires the DDH to hold at every level and without it the system will be vulnerable to being completely exposed.

3. Compare the communication complexity of TGDH (number of messages and blinded keys broadcast) for Join, Leave, and Merge events against a naive approach where all n members perform pairwise Diffie-Hellman key exchanges.

The cost to create a naive approach with Diffie-Hellman Key exchanges would be $\frac{n(n-1)}{2}$ as every member who is joining would have to do a DH key exchange with every other member resulting in an $O(n^2)$ time complexity. So, when one member joins or leaves, they would have to make a key exchange or leave an exchange resulting in a time complexity of $O(n)$. When attempting to merge two groups, it is similar to the initial set up where every

member from each group must make a new key exchange with every member from the other group resulting in a time complexity of $O(n^2)$.

For the TGDH, the cost to join and leave will correspond to the height of the current tree. This is because only the path to the root of the tree needs to update the blinded keys to broadcast. This will result in a time complexity of $O(h)$ where $h = \log(n)$. In order to merge two trees, each member will only have to broadcast their keys once, resulting in a time complexity of $O(n)$ because it is a linear operation. In TGDH, the number of messages and blinded broadcast keys are the same for each event.

	TGDH (messages)	TGDH (broadcast)	Naive Approach
Join	$O(\log(n))$	$O(\log(n))$	$O(n)$
Merge	$O(n)$	$O(n)$	$O(n^2)$
Leave	$O(\log(n))$	$O(\log(n))$	$O(n)$